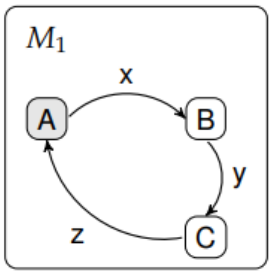
IS1 - Seguridad, Vitalidad y Equidad

1 Pasos repetitivos (stuttering steps)

Consideremos el sistema descripto por la siguiente máquina de estados, donde el estado inicial es el gris (A) y la semántica del sistema se define como el conjunto de todas las sucesiones infinitas de estados que surgen de ejecutar las transiciones.



Si el sistema permanece infinito tiempo en un estado, éste se repite al infinito en la sucesión. A una sucesión infinita de estados la llamaremos *ejecución* o *comportamiento*.

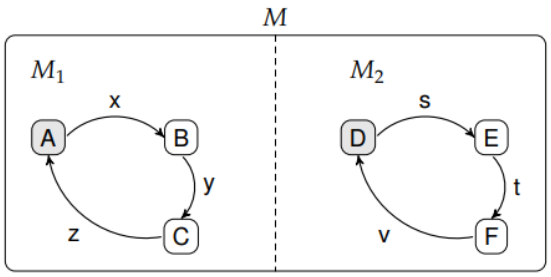
Algunas de las ejecuciones de M_1 son las siguientes (donde $\widehat{\cdot}$ significa que $\cdot$ se repite al infinito):

$$\langle \widehat{A} \rangle \tag{1}$$

$$\langle A, B, \widehat{C} \rangle \tag{2}$$

$$\langle A, B, C, \widehat{A} \rangle \tag{3}$$

Consideremos el sistema M que resulta de componer en paralelo M_1 y M_2.



Algunas ejecuciones son:

$$\langle \widehat{(A, D)} \rangle \tag{4}$$

$$\langle (A, D), (B, D), (B, E), \widehat{(C, E)} \rangle \tag{5}$$

Consideremos la proyección de (5) a M_1:

$$\mathcal{P}_{M_1}(\langle (A, D), (B, D), (B, E), \widehat{(C, E)} \rangle) = \langle A, B, B, \widehat{C} \rangle \tag{6}$$

El problema con (6) es que no es una de las ejecuciones admitidad por M_1, puesto que B se repite un número finito de veces en el medio de la ejecución, mientras que en todas las ejecuciones de M_1, si B se repite lo hace infinitas veces y al final de la ejecución.

Los **pasos repetitivos** (stuttering steps) permiten evitar esta inconsistencia. Definimos una **ejecución** de un cierto sistema M como el conjunto de sucesiones infinitas de estados que surgen de ejecutar las transiciones pero donde se admitan *repeticiones finitas de estados* llamadas *pasos repetitivos*.

Normalmente los pasos repetitivos no se explicitan sino que se asume que cada estado puede repetirse un número finito de veces (por ejemplo en (2), A y B pueden repetirse un número finito de veces).

2 Concepto de estado

Sea Var el conjunto de todos los nombres posibles de variables y sea Val la colección de todos los valores que las variables pueden tomar. Entonces el **estado** s se define como una función de Var en Val:

$$s : Var \to Val$$

Definir el concepto de estado de esta forma surge de la necesidad de poder especificar un sistema como la composición de las especificaciones de subsistemas o componentes. Cualquier estado depende potencialmente de todas las variables posibles solo que cada componente especifica ciertas condiciones para algunas de las variables.

Al definir el concepto de estado de esta forma estamos definiendo el *estado de un universo discreto*.

Supongamos que el sistema M_1 depende de la variable de estado x que toma valores en $\mathbb{N}$. La única transición permitida en M_1 es $T_{M_1} \triangleq x' = x + 1$ y el estado inicial de M_1 es $x = 0$. Entonces las ejecuciones de M_1 tienen la siguiente forma:

$$\langle x = 0, x = 1, x = 2, \dots \rangle$$

Ahora supongamos que tenemos otro sistema M_2 que depende de la variable de estado y que toma valores en $\mathbb{N}$. La única transición permitida en M_2 es $T_{M_2} \triangleq y' = y + 2$ y el estado inicial de M_2 es $y = 0$. Entonces las ejecuciones de M_2 tienen la siguiente forma:

$$\langle y = 0, y = 2, y = 4, \dots \rangle$$

Supongamos que el sistema M es el resultado de componer M_1 con M_2 de forma tal que la única transición permitida es $T_M \triangleq T_{M_1} \vee T_{M_2}$. Si s es un estado del sistema M_1 o de M_2 o de M, tanto $s(x)$ como $s(y)$ están definidas. Por lo tanto, si $e \triangleq \langle e(0), e(1), e(2), \dots \rangle$ es una ejecución, lo es de M_1 sí y solo si

$$e(0)(x) = 0 \wedge e(1)(x) = 1 \wedge e(2)(x) = 2 \wedge \dots$$

y lo es de M_2 sí y solo si

$$e(0)(y) = 0 \wedge e(1)(y) = 2 \wedge e(2)(y) = 4 \wedge \dots$$

3 Teorema de Alpern-Schneider

Consideraremos que una **propiedad** de un sistema concurrente es un conjunto de ejecuciones. Un sistema concurrente queda caracterizado por el conjunto de ejecuciones que admite. Decimos que un sistema M verifica una propiedad P sii $M \subseteq P$, es decir, si el conjunto de ejecuciones que caracteriza a M está incluido o es igual al conjunto de ejecuciones que defone la propiedad P.

Algunas propiedades que se pueden expresar como conjuntos de ejecuciones son deadlock freedom, corrección parcial, exclusión mutua, starvation freedom, terminación y garantía de servicio.

Teorema de Alpern-Schneider. *Toda propiedad P es el resultado de la intersección de una propiedad de seguridad (safety) con una propiedad de vitalidad (liveness).*

El teorema implica que todo sistema puede expresarse como la intersección de una propiedad de seguridad (*safety*) con una propiedad de vitalidad (*liveness*).

3.1 Seguridad (Safety)

Las propiedades de seguridad postulan que *nada malo puede pasar* durante una ejecución del sistema. Toda propiedad de seguridad *proscribe* o *prohíbe* algo malo. Algunos ejemplos son deadlock freedom, correción parcial, exclusión mútua y fcfs.

La forma más fácil de dar una propiedad de seguridad es indicando sólo las transiciones de estado que están permitidas. Una propiedad de seguridad normalmente se da indicando sólo los comportamientos permitidos e implícitamente se prohíben todos los demás. Una propiedad de seguridad *no* indica que el sistema hará algo, sólo indica lo que tiene permitido hacer.

Sucesiones. *Sea E un conjunto de estados. E^* denota el conjunto de todas las sucesiones finitas de elementos de E (llamadas ejecuciones parciales), y E^∞ el de todas las sucesiones infinitas (las ejecuciones). Si $e \in E^\infty$ y $n \in \mathbb{N}$, entonces e_n denota el prefijo de e hasta n y $e(n)$ el n-ésimo elemento de la sucesión (siendo $e(0)$ el primer elemento). Es decir, $e_n = \langle e(0), e(1), \ldots, e(n)\rangle$.*

Un prefijo de una ejecución es una ejecución parcial. Si e es una ejecución parcial y t es una sucesión de estados, $e \circ t$ denota la concatenación de e con t. Si e es una ejecución parcial, e^∞ es la ejecución que resulta de repetir al infinito el último estado de e.

Si P es una propiedad y e una ejecución parcial, definimos que $e \in P$ sii $e^\infty \in P$.

Seguridad. *S es una propiedad de **seguridad** respecto de E sii se verifica que $e \in S \Leftrightarrow \forall\, n \in \mathbb{N} : e_n \in S$ para toda $e \in E^\infty$.*

S es una propiedad de seguridad cuando una ejecución pertenece a S sii todos sus prefijos pertenecen a S. Las siguientes son definiciones equivalentes $\forall\, e \in E^\infty$

1. $e \notin S \Leftrightarrow \exists\, n \in \mathbb{N} : \forall\, t \in E^\infty : e_n \circ t \notin S$
2. $e \notin S \Leftrightarrow \exists\, n \in \mathbb{N} : e_n \notin S$

Si S es de seguridad y $e \notin S$ entonces hay un estado en e donde se viola la propiedad de seguridad, es decir, en algún e_n ocurre lo "malo" y eso luego no se puede remediar.

Las propiedades de seguridad son *discretas* en el sentido de que hay un punto preciso donde se puede ver la violación de la propiedad.

3.2 Vitalidad (Liveness)

Las propiedades de vitalidad estipulan que algo *bueno* ocurrirá durante la ejecución del sistema. Toda propiedad de vitalidad asegura que algo eventualmente ocurrirá en el sistema. Algunos ejemplos son ausencia de inanición, terminación y garantía de servicio.

Una propiedad de vitalidad dice que ninguna ejecución parcial es irremediable: siempre existe la posibilidad de que más adelante la ejecución cumpla con la propiedad, es decir, indica los estados que obligatoriamente deberán alcanzarse durante una ejecución.

Vitalidad. *L es una propiedad de **vitalidad** respecto de E sii se verifica que $\forall\, e \in E^* : \exists\, t \in E^\infty : e \circ t \in L$.*

L es una propiedad de vitalidad si toda ejecución parcial puede ser extendida a una ejecución donde lo *bueno* estipulado por L ocurre. Un sistema verifica L si toda ejecución parcial del sistema puede ser extendida a una ejecución donde lo estipulado por L se da.

No necesariamente hay un punto preciso donde la propiedad de vitalidad se haga verdadera. Ciertas propiedades requieren que muchos estados la verifiquen (por ejemplo, ausencia de inanición), por lo que las propiedades de vitalidad no son discretas. Una propiedad de vitalidad no estipula que algo bueno *siempre* ocurre, solo indica que *eventualmente* ocurrirá.

Supongamos que L es de vitalidad pero no verifica la definición. Entonces existe una ejecución parcial e tal que $\forall\, t \in E^\infty : e \circ t \notin L$, lo cual sugiere que e es algo prohibido por L por lo que L es, en parte, de seguridad. Al relajar la definición de vitalidad se obtienen propiedades que prohiben cosas (por lo que serían de seguridad). Por lo tanto se considera que la definición de vitalidad incluye a todas las propiedades puramente de vitalidad.

3.3 Máquinas cerradas

No necesariamente todas las restricciones de un sistema están dadas por la propiedad de seguridad, es decir, no podemos decir si una transición está permitida o no mirando solo la propiedad de seguridad. Las especificaciones donde sucede esto son inapropidadas. El problema se origina al permitir propiedades de vitalidad arbitrarias.

Clausura. *Si A es una propiedad cualquiera, $\mathcal{C}(A)$, denominada* ***clausura de*** *A, es la menor propiedad de seguridad que contiene a A.*

Máquina Cerrada. *Si S es una propiedad de seguridad y L es una propiedad cualquiera, entonces el par (S, L) es una* ***máquina cerrada*** *sii $\mathcal{C}(S \cap L) = S$.*

Si (S, L) es una máquina cerrada, todas las restricciones del sistema definido por $S \cap L$ están dadas por S ya que no existe S' de seguridad tal que $S \cap L \subseteq S' \subset S$. Si tal S' existiera querría decir que L agrega restricciones.

Si las propiedades de vitalidad se escriben como propiedades de *equidad* entonces se obtienen máquinas cerradas.

4 Equidad (Fairness)

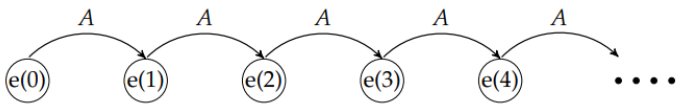
Las propiedades de **equidad** (fairness) son una clase de vitalidad con una forma particular.

Sea P un predicado de estado (es decir, un predicado que depende de las variables de estado), entonces $\Box P$ (que se lee *siempre* P) es un predicado de la lógica temporal que depende de una ejecución: $\Box P$ es verdadero en $e \in E^\infty \Leftrightarrow \forall\, i \in \mathbb{N} : P(e(i))$.

Esta es una forma temporal de expresar que P es un invariante de estado en la ejecución e.

Sea A un predicado que depende de dos estados (es decir, A es una transición de estados), entonces $\Box A$ es un predicado de la lógica temporal que depende de una ejecución: $\Box A$ es verdadero en $e \in E^\infty \Leftrightarrow \forall\, i \in \mathbb{N} : A(e(i), e(i+1))$.

En este caso el sistema pasa del estado $e(i)$ al $e(i+1)$ siempre debido a A. Cuando es así decimos que se da un *A-step*.



Si F es un predicado cualquier, $\Diamond F$ se define como $\neg\, \Box \neg\, F$ y se lee *eventualmente* F. Formalmente, $\Diamond F$ es verdadero en $e \in E^\infty \Leftrightarrow \exists\, i \in \mathbb{N} : F(e(i))$ o $F(e(i), e(i+1))$ si F es una transición.

Existen dos combinaciones importantes de $\Box$ y $\Diamond$:

- $\Diamond\Box F$ se lee *eventualmente siempre* F y es verdadero si existe un estado a partir del cual F es siempre verdadero.
- $\Box\Diamond F$ se lee *infinitamente a menudo* F y es verdadero si existen infinitos estados donde F es verdadero.

Si A es una transición de estados, el predicado $enabled(A)$ es verdadero cuando la precondición de A se satisface (es decir, cuando es posible ejecutar la transición A o cuando A está *habilitada* para ejecutar).

Existen dos versiones de equidad: *débil* (weak fairness) y *fuerte* (strong fairness).

Weak Fairness. *La transición de estados A verifica* ***weak fairness*** *sii $WF(A) \triangleq \Diamond\Box enabled(A) \Rightarrow \Box\Diamond A$.*

Strong Fairness. *La transición de estados A verifica* ***strong fairness*** *sii $SF(A) \triangleq \Box\Diamond enabled(A) \Rightarrow \Box\Diamond A$.*

En WF se pide que a partir de cierto punto A esté siempre habilitada, mientras que en SF es suficiente con que A esté habilitada en infinitos estados aunque puede no estarlo de forma continua. En cualquiera de los dos casos se exige que A sea ejecutada infinitas veces.

Teorema de Abadi-Lamport. *Si S es una propiedad de seguridad y L es una conjunción numerable de fórmulas $WF(A_i)$ o $SF(A_i)$ tales que cada $A_i\,(i \in I)$ puede ejecutarse satisfaciendo S, entonces (S, L) es máquina cerrada.*

Una forma de cumplir con la condición "A_i puede ejecutarse satisfaciendo S" es definiendo S como $\bigvee_{i \in I} A_i$. Es decir, S es la disyunción de todas las transiciones permitidas, lo que además implica que S es de seguridad.